

Compiler Explorer links

One link per demo, preconfigured for x86-64 GCC 14.2 with `-std=c++23 -O2 -Wall -Wextra -Wpedantic` and an execution pane, so the program's output appears under the assembly. Click **run**; where a demo has deliberate errors behind `SHOW_ERRORS`, **errors** opens the same file with `-DSHOW_ERRORS` so you can read the diagnostics. Generated by `tools/godbolt-links.py`; do not edit by hand.

Demos that need more than one file or a library godbolt does not provide (demos/s05/modules, demos/s04/parallel.cpp) are built from the repo instead.

Session 1

Demo	What it shows	Open
auto_pitfalls.cpp	auto pitfalls (C++11 calibration)	run
copy_elision.cpp	guaranteed copy elision (C++17)	run
ctad.cpp	class template argument deduction (C++17)	run
designated_init.cpp	designated initializers (C++20)	run
escaping_lambda.cpp	lambdas that outlive their captures (C++11 calibration)	run
from_chars.cpp	<code>std::from_chars</code> replacing <code>strtod</code> (C++17)	run
generic_lambda.cpp	generic lambdas and init-capture (C++14)	run
inline_variable.cpp	inline variables (C++17)	run
move_does_nothing.cpp	when <code>std::move</code> does nothing (C++11 calibration)	run · errors
nodiscard.cpp	<code>[[nodiscard]]</code> (C++17), <code>[[maybe_unused]]</code> and <code>[[fallthrough]]</code> (C++17)	run · errors
range_for_init.cpp	range-for with initializer (C++20)	run
small_cpp23.cpp	the small C++23 features (C++23)	run
spaceship.cpp	three-way comparison and defaulted comparisons (C++20)	run
spaceship_details.cpp	three-way comparison details (C++20)	run
structured_bindings.cpp	structured bindings and if-with-initializer (C++17)	run
using_enum.cpp	using enum (C++20)	run

Session 2

Demo	What it shows	Open
any.cpp	<code>std::any</code> (C++17)	run

expected_basics.cpp	<code>std::expected</code> (C++23)	run
expected_pipeline.cpp	monadic <code>std::expected</code> (C++23)	run
expected_void.cpp	<code>expected<void, E></code> (C++23)	run
format_ranges.cpp	formatting ranges (C++23)	run
format_specs.cpp	<code>std::format</code> and the spec mini-language (C++20)	run
format_to_buffer.cpp	<code>format_to</code> , <code>format_to_n</code> , <code>formatted_size</code> (C++20)	run
formatter_custom.cpp	<code>std::formatter</code> for your own types (C++20)	run
formatter_spec.cpp	a formatter that accepts its own format spec (C++20)	run
optional_basics.cpp	<code>std::optional</code> (C++17)	run
optional_monadic.cpp	monadic <code>optional</code> (C++23)	run
print.cpp	<code>std::print</code> and <code>std::println</code> (C++23)	run
span_basics.cpp	<code>std::span</code> (C++20)	run
span_bytes.cpp	<code>span</code> for raw memory (C++20, <code>std::byte</code> C++17)	run
string_view_basics.cpp	<code>std::string_view</code> (C++17)	run
string_view_dangling.cpp	the four ways a <code>string_view</code> dangles (C++17)	run · errors
string_view_not_cstring.cpp	a <code>string_view</code> is not a C string (C++17)	run
tour_bit.cpp	and <code>std::source_location</code> (C++20)	run
tour_chrono.cpp	calendars and time zones (C++20)	run
tour_cpp17.cpp	C++17 library odds and ends	run
tour_cpp20.cpp	C++20 library odds and ends	run
tour_cpp23.cpp	C++23 library odds and ends	run
tour_filesystem.cpp	<code>std::filesystem</code> (C++17)	run
tour_flat_map.cpp	<code>std::flat_map</code> (C++23)	run
tour_stacktrace.cpp	<code>std::stacktrace</code> (C++23)	run
transparent_compare.cpp	transparent comparators and heterogeneous lookup (C++14 <code>map</code> , C++20 <code>unordered_map</code>)	run
variant_messages.cpp	variant as a closed message set (C++17)	run
variant_visit.cpp	<code>std::variant</code> and <code>std::visit</code> (C++17)	run

Session 3

Demo	What it shows	Open
auto_nttp.cpp	auto non-type template parameters (C++17)	run

	+17) and class-type NTTPs (C++20)	
concepts_basics.cpp	the four ways to constrain a template (C++20)	run · errors
concepts_vs_sfinae.cpp	the error messages, side by side (C++11 vs C++20)	run · errors
constexpr_alloc.cpp	constexpr and constinit (C++20)	run
constexpr_evolution.cpp	compile-time std::vector and std::string (C++20)	run
constexpr_limits.cpp	the same computation under each standard's constexpr rules	run
constexpr_limits.cpp	what a constant expression refuses (and why that is a feature)	run · errors
ctad_guides.cpp	CTAD and deduction guides (C++17), aggregate CTAD (C++20)	run
deducing_this.cpp	deducing this (C++23)	run
fold_expressions.cpp	fold expressions (C++17)	run
if_constexpr_dispatch.cpp	if constexpr replacing tag dispatch and enable_if (C++17)	run
is_constant_evaluated.cpp	std::is_constant_evaluated (C++20) and if constexpr (C++23)	run · errors
requires_expressions.cpp	requires expressions (C++20)	run
subsumption.cpp	subsumption, how the more constrained overload wins (C++20)	run
template_lambdas.cpp	template lambdas and lambdas in unevaluated contexts (C++20)	run
variable_templates.cpp	variable templates (C++14) and _v/_t aliases (C++17)	run

Session 4

Demo	What it shows	Open
adaptor_closure.cpp	storing and composing adaptors; your own adaptor (C++20 / C++23)	run
adaptors_tour.cpp	the core view adaptors (C++20)	run
algorithm_results.cpp	what constrained algorithms return (C++20)	run
chunk_by.cpp	views::chunk_by, the group-by (C++23)	run
const_view_trap.cpp	the const view trap (C++20)	run · errors
constrained_algorithm	std::ranges:: algorithms vs std:: algorithms	run

s.cpp	(C++20)	
cost_model.cpp	the cost model of views (C++20)	run
cpp23_algorithms.cpp	C++23 constrained algorithm additions	run
dangling.cpp	borrowed ranges and dangling (C++20)	run
fold.cpp	ranges::fold_left and friends (C++23)	run
lazy.cpp	views are lazy (C++20)	run
numeric17.cpp	C++17 numeric algorithms	run
parallel.cpp	parallel algorithms (C++17 execution policies)	from the repo
projections.cpp	projections (C++20)	run
range_concepts.cpp	the range concept hierarchy (C++20)	run
ranges_to.cpp	std::ranges::to (C++23)	run
same_task_three_ways.cpp	the same task in C++11, C++20, and C++23	run
sentinels.cpp	sentinels (C++20)	run
windows.cpp	chunk, slide, stride, adjacent (C++23)	run
zip_enumerate.cpp	zip, enumerate, and iota for indexing (C++23)	run
zip_family.cpp	cartesian_product, repeat, join_with, as_rvalue (C++23)	run

Session 5

Demo	What it shows	Open
atomic_shared_ptr.cpp	std::atomic (C++20): hot-swappable configuration	run
atomic_wait.cpp	atomic wait/notify and atomic_ref (C++20)	run
awaitable.cpp	co_await and a hand-written awaitable (C++20)	run
coroutine_machinery.cpp	what the compiler generates for a coroutine (C++20)	run
deprecated.cpp	deprecations and removals, C++14 through C++23	run · errors
false_sharing.cpp	hardware_destructive_interference_size (C++17) and false sharing	run
generator_basics.cpp	std::generator (C++23)	run
generator_perf.cpp	what a generator costs (C++23)	run
jthread.cpp	std::jthread (C++20)	run
latch_barrier.cpp	std::latch and std::barrier (C++20)	run
move_only_function_q	a task queue with std::move_only_function	run

ueue.cpp	(C++23)	
osyncstream.cpp	std::osyncstream (C++20)	run
scoped_lock.cpp	std::scoped_lock (C++17)	run
semaphore_queue.cpp	a bounded queue, C++11 vs C++20 (counting_semaphore)	run
shared_mutex.cpp	shared_timed_mutex (C++14), shared_mutex (C++17): reader/writer locks	run
state_machine.cpp	a coroutine as a state machine (C++20)	run
stop_token.cpp	stop_token, stop_source, stop_callback (C++20)	run
tsan_race.cpp	a data race ThreadSanitizer catches (build with -DCOURSE_SANITIZE=thread)	run · errors

The exercise programs

The whole telemetry processor, one link per starter and solution, flattened to a single file with data/sample.csv on stdin: the report appears in the output pane. The Session 1 starter is compiled as C++11, everything else as C++23. The repo's test suites do not run here; use these to read and tweak the program, not to grade it.

Exercise	What it asks	Open
s01-modernize-syntax	the telemetry processor in careful C++11; modernize the syntax without changing the report	starter · solution
s02-vocabulary-types	replace out-parameters and nullable pointers with optional, expected, string_view and span	starter · solution
s03-compile-time	move the CRC table and the sensor-config checks to compile time; constrain the templates	starter · solution
s04-ranges	replace the hand-written loops with constrained algorithms and views; the report stays byte-identical	starter · solution
s05-concurrency	split loading and statistics across a producer and a consumer thread; then a generator	starter · solution